EEEM071 Advanced Topics in Computer Vision and Deep Learning Lab Week 5

Introduction to Python

Dr. Xiatian Zhu
xiatian.zhu@surrey.ac.uk

1 Introduction

Intended Audience: We assume you now have a basic hands-on experience in programming in MATLAB. The objective of this lab (and the upcoming sessions) is to introduce you to the Python programming language which will eventually assist you in completing your coursework.

Many of you might already have some prior experience using Python and its libraries for instance, `numpy`, `scipy` etc. (as a part of EEEM068 or elsewhere). Hence, please feel free to rather have a quick glance through this notebook, and jumpstart with your coursework project.

Happy Programming! Please open this colab notebook. Follow the instruction in the lab sheet week 5-10 to learn how to copy to your google drive.

2 Python Programming Language

An interpreted, Object-oriented, garbage-collected programming language.

“interpreted": Two types of languages -interpreted or compiled. Former, compiles (translates) the entire code into a machine-level code before execution, whereas the latter tends to translate each line one by one at run-time and hence is comparatively slower but supports platform independence since binary code is not pre-compiled.

“Object-oriented": Provision for Classes and Objects and hence supports all fundamental OOPs concepts - encapsulation, abstraction, inheritance, and polymorphism.

“Garbage-collected": Consists of a dedicated engine, which frees up memory for objects no longer needed by the program (have gone out-of-scope).

2.1 Hello World!

A “Hello, World!" program is a simple computer program that outputs the text “Hello, World!" on the screen or console. It is typically the first program that developers create when they are learning a new programming language, as it demonstrates the basic syntax and structure of the language. Here’s an example of a "Hello, World!" program in Python:

Instruction: Run the code in the section **Hello World** of the Colab notebook.

The `print()` command is a function in most programming languages that allows the output of text or data to the console or screen. It is used to display a message or a value on the screen during program execution.

The `print()` command usually takes one or more arguments, which can be text strings or variables. The arguments are separated by commas. When the `print()` function is called, the arguments are concatenated into a single string and then displayed on the console or screen.

2.2 Data Types

In programming, a data type is an attribute of data that defines the kind of value that a variable or constant can hold. It specifies the type of data that a programming language supports such as integers, floating-point numbers, strings, characters, booleans, arrays, etc. Each programming language has its own set of data types that it supports.

Data types play an important role in programming because they determine the operations that can be performed on the data. For example, arithmetic operations can only be performed on numeric data types, while string manipulation operations can only be performed on string data types.

There are four main data types in Python:

1. Integer (`int`) - All integer values.
2. Float (`float`) - All floating point decimal values.
3. String (`str`) - Strings or single characters.
4. Boolean (`bool`) - True/False

Instruction: Run the code in the section **Datatypes** of the colab notebook.

Knowing the data types and their respective properties is crucial for writing accurate and efficient programs.

2.3 Typecasting

Typecasting, also known as type conversion, is the process of converting a value from one data type to another in a programming language. This is necessary when performing operations between values of different data types or when assigning a value of one data type to a variable of another data type.

Python provides a convenient way to typecast or change existing datatypes using constructor functions: int(), str(), float()

Instruction: Run the code in the section **Typecasting** of the colab notebook.

In Python, if you add an integer and a floating-point number, the result will be a floating-point number because the language automatically converts the integer to a float before performing the addition. This is called *implicit typecasting*, such as when a mathematical operation is performed between values of different data types. Try it yourself!

2.4 Containers (or Collections)

In programming, a container or collection is an object that holds a group of other objects or values. The container provides a way to store and organize multiple values under a single variable name. Containers are used to manage and manipulate groups of data in a program.

There are several types of containers or collections available in most programming languages. Some of the most common container types include:

1. Lists - `list([..., ])`: A list is a collection of values that are ordered and changeable. Lists are typically used for storing multiple items of the same data type.
2. Tuples - `tuple(..., )`: A tuple is similar to a list, but it is immutable, meaning that its contents cannot be changed once it has been created. Tuples are typically used for storing multiple items of different data types.
3. Sets- `set(..., )`: A set is a collection of unique values that are unordered and unindexed. Sets are typically used for performing operations such as union, intersection, and difference on two or more sets.
4. Dictionaries - `dict(key=value, ..., )`: A dictionary is a collection of key-value pairs that are unordered and changeable. Dictionaries are typically used for storing data in a way that can be easily accessed and manipulated by the program.

Instruction: Run the code in the section **Containers (or Collections)** of the colab notebook.

Containers are important in programming because they allow you to store, access, and manipulate large amounts of data efficiently. By organizing data into containers, you can make your code more readable, maintainable, and scalable.

2.5 Condtional Statements

In programming, conditional statements are statements that allow a program to make decisions and execute different code based on whether a condition is true or false. Conditional statements are a fundamental concept in programming and are used to add logic and control flow to programs.

There are typically two types of conditional statements:

If statements: An if statement is a type of conditional statement that allows a program to execute code only if a specified condition is true. If the condition is false, the program will skip the code block.

If-else statements: An if-else statement is a type of conditional statement that allows a program to execute one code block if a specified condition is true, and another code block if the condition is false.

Instruction: Run the code in the section **Conditional Statements** of the colab notebook.

2.6 Control Statements - break, continue

In programming, control statements are statements that are used to control the flow of a program. Two common control statements used in loops are break and continue.

`break`: The break statement is used to exit a loop early. When the break statement is encountered, the loop is immediately terminated, and the program continues executing the code that follows the loop.

`continue`: The continue statement is used to skip over the current iteration of a loop and continue with the next iteration. When the continue statement is encountered, the program immediately jumps to the next iteration of the loop, skipping over any remaining code in the current iteration.

Instruction: Run the code in the section **Control Statements - break, continue** of the colab notebook.

2.7 Loops

In programming, loops are used to repeatedly execute a block of code until a certain condition is met. Loops are a fundamental programming concept and are used to iterate over data structures, perform calculations, and automate repetitive tasks.

There are typically two types of loops:

`for` loop: A for loop is used to iterate over a sequence of values, such as a list, tuple, or string. The loop will execute the code block for each value in the sequence.

`while` loop: A while loop is used to repeatedly execute a code block as long as a certain condition is true. The loop will continue to execute until the condition is false.

Instruction: Run the code in the section **Loops** of the colab notebook.

Loops are essential in programming because they allow you to automate repetitive tasks and iterate over data structures. They can help you write more efficient and scalable code and are used in many programming languages and applications.

2.8 Functions

In programming, a function is a block of code that performs a specific task or a set of tasks. Functions are a fundamental programming concept and are used to organize code into modular and reusable blocks.

Functions can be called by other parts of the program, and they can take input parameters and return output values. When a function is called, the program will execute the code within the function and then return control to the calling code.

Instruction: Run the code in the section **Functions** of the colab notebook.

Functions are an important tool in programming because they allow you to break down complex problems into smaller, more manageable tasks. By organizing code into functions, you can make your code more modular, reusable, and easier to understand and maintain.

2.9 Classes and Objects

In programming, a class is a blueprint or template for creating objects that share common properties and behaviors. An object, on the other hand, is an instance of a class that has its own unique state and behavior.

Classes are a fundamental programming concept in object-oriented programming (OOP), which is a programming paradigm that focuses on creating objects that interact with each other to perform tasks. OOP is based on the concepts of encapsulation, inheritance, and polymorphism.

Instruction: Run the code in the section **Classes and Objects** of the colab notebook.

Classes and objects are essential tools in programming because they allow you to create complex systems that can be easily managed and extended. By defining classes and creating objects, you can organize your code into logical units, abstract away implementation details, and create reusable components that can be used across multiple projects.

2.10 Modules and Packages

In Python, a module is a file containing Python code. A module can define functions, classes, and variables, which can be used in other Python programs. When you want to use code from a module in your program, you can import the module using the import statement.

A package is a collection of modules that are organized in a directory hierarchy. Packages are used to organize related modules and provide a namespace to avoid naming conflicts. A package can contain subpackages, which are themselves packages, and modules.

Instruction: Run the code in the section **Modules and Packages** of the colab notebook.

When you install Python packages using a package manager like `pip`, the packages are downloaded and installed from a repository of pre-built packages called the Python Package Index (PyPI). There are thousands of third-party packages available on PyPI that you can use to extend Python's functionality, ranging from data analysis libraries like `pandas` and `numpy`, to image processing libraries like `opencv` and `scikit-image`, to deep learning frameworks like `tensorflow` and `pytorch`, and to web frameworks like `Django` and `Flask`.

3 References

1. https://www.learnpython.org/
2. https://www.tutorialspoint.com/python/index.htm
3. https://www.w3schools.com/python/